

# Ejercicios Prácticos: Listas y Ciclos

Ejercicios de Aplicación en Python

Clase de Programación

Profesor: Mateo Taito Stambuk

Email: [mateo.taitos@edu.uai.cl](mailto:mateo.taitos@edu.uai.cl)

27 de Mayo, 2026

# Objetivos de la Clase

- **Repasar** el uso de índices para iterar sobre listas
- **Relacionar** datos almacenados en listas paralelas (unidimensionales)
- **Aplicar** ciclos anidados para problemas simples en 2D
- **Desarrollar** lógica para imprimir figuras en pantalla (patrones)

# 1 Listas Paralelas por Índice

# Ejercicio 1: El Mejor Promedio

## Enunciado del Problema

Tenemos dos listas con información sobre estudiantes:

1. Una lista con los **nombres** de los alumnos.
2. Una lista con sus **calificaciones finales** (en el mismo orden).

Misión:

1. Encontrar el alumno con la **mejor calificación**.
  2. Mostrar su nombre y su nota en pantalla.
- **Restricción:** No se pueden usar funciones avanzadas como `zip()` o `.index()`.
  - Debemos recorrer las listas usando sus **índices**.

# Ejercicio 1: Planteamiento

Tenemos las siguientes listas iniciales:

```
nombres = ["Ana", "Juan", "Pedro", "Sofía", "Luis"]  
notas = [4.5, 6.2, 5.8, 6.9, 5.0]
```

La relación por índice:

- El índice `0` corresponde a Ana (nota 4.5).
- El índice `1` corresponde a Juan (nota 6.2).
- Y así sucesivamente...

Estrategia:

- Definir variables para guardar la `nota_maxima` y el `mejor_alumno`.
- Usar un `for` iterando desde `0` hasta el largo de la lista.
- Comparar cada nota y actualizar si encontramos una mayor.

# Ejercicio 1: Código Paso a Paso

```
# 1. Definimos las listas
nombres = ["Ana", "Juan", "Pedro", "Sofía", "Luis"]
notas = [4.5, 6.2, 5.8, 6.9, 5.0]

cantidad_alumnos = len(nombres)

# 2. Inicializamos variables (buscando un máximo)
mejor_nota = -1.0
mejor_alumno = ""

# 3. Recorremos usando índice
for i in range(cantidad_alumnos):
    if notas[i] > mejor_nota:
        mejor_nota = notas[i]
        mejor_alumno = nombres[i]

# 4. Resultado
print(f"El mejor alumno es {mejor_alumno} con un {mejor_nota}")
```

# Ejercicio 1: Explicación Detallada

- Línea 5: Usamos `len(nombres)` para saber cuántos elementos hay. Sirve para ambas porque miden lo mismo.
- Líneas 8-9: Iniciamos `mejor_nota` en `-1.0`. Así nos aseguramos de que el primer valor real siempre será mayor.
- Línea 12: `range(cantidad_alumnos)` genera la secuencia `0, 1, 2, 3, 4`.
- Líneas 13-15: Comparamos y, si la nota actual es mayor, sobrescribimos nuestros registros.
- ¡El truco está en usar el mismo índice (`i`) para acceder a la nota (`notas[i]`) y al nombre correspondiente (`nombres[i]`).

2

## Impresión de Figuras

# Ejercicio 2: Triángulo Rectángulo Invertido

## Enunciado del Problema

Se pide crear un programa que dibuje un triángulo rectángulo en la pantalla utilizando el carácter asterisco ( \* ).

El usuario define (o dejamos en código duro) la **altura**. Si la altura es 5 , se debe ver así:

```
* * * * *  
* * * *  
* * *  
* *  
*
```

# Ejercicio 2: Planteamiento

Para dibujar en consola, imprimimos línea por línea (de arriba hacia abajo).

- Fila 1: 5 asteriscos
- Fila 2: 4 asteriscos
- Fila i: `altura - i + 1` asteriscos (o iteramos en reversa)

Dos enfoques posibles:

1. **Uso de ciclos anidados:** Un ciclo para las filas y otro interno para los asteriscos (enfoque universal).
2. **Multiplicación de strings:** Multiplicar `*` por el número de la fila (atajo de Python).

# Ejercicio 2: Código con Ciclos Anidados

Este es el enfoque clásico, que te servirá si en el futuro programas en C, Java o C#.

```
altura = 5
print("Método 1: Ciclos Anidados")

for fila in range(altura, 0, -1):
    for columna in range(fila):
        print("* ", end="") # end="" evita el salto de línea
    print() # Salto de línea al terminar la fila
```

- Línea 4: `range(altura, 0, -1)` itera en reversa desde `5` hasta `1`.
- Línea 5: El ciclo interno depende de la `fila` actual. En la primera iteración, al valer `5`, este ciclo se ejecuta 5 veces.
- Línea 7: El `print()` vacío genera el enter/salto a la siguiente línea del terminal una vez que terminamos de dibujar esa fila.

# Ejercicio 2: El atajo de Python

En Python, podemos simplificar el proceso porque los "strings" se pueden multiplicar por un entero.

```
altura = 5
print("Método 2: Multiplicación de Strings")

for fila in range(altura, 0, -1):
    print("* " * fila)
```

- La magia de `"* " * 3` dará como resultado la cadena combinada `"* * * "`.
- Esto nos permite evitar el segundo ciclo ( `for` interno), haciendo el código mucho más simple.
- Dato: Es importante conocer ambos. Las lógicas de multiplicar strings no siempre existen en otros lenguajes.

# ¡Gracias!

¿Preguntas sobre estos ejercicios?